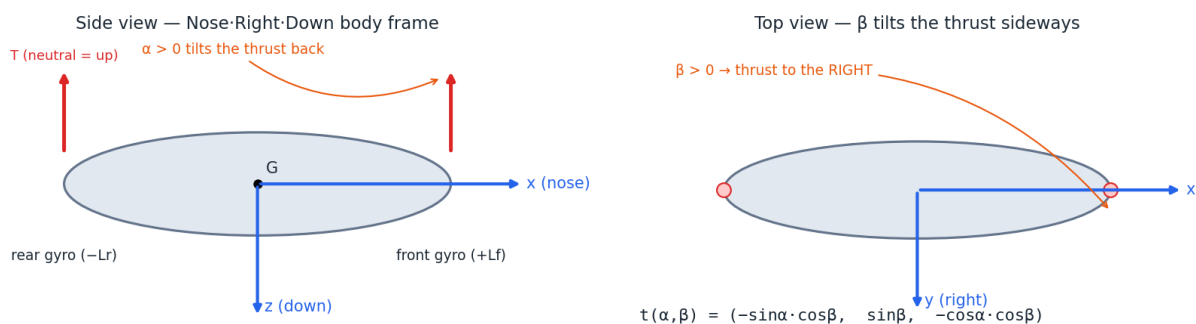


03/07/2026 · solution repository: github.com/kingsteagall/hylight-gnc-test

Exercise 2 — drift diagnosis from the Annex B log, and a course-based guidance fix, measured: ~3× less cross-track error on two independent simulations. Full sources in the repository above.

Each gyro is a thrust vector on a 2-axis gimbal: the exterior servo (α , the outer gimbal — it carries the other one) tilts the thrust fore/aft; the interior servo (β) tilts it left/right. Neutral points up. Composing the two rotations, outer applied last:



$$t(\alpha, \beta) = R_y(\alpha) \cdot R_x(\beta) \cdot (0, 0, -1) = (-\sin \alpha \cdot \cos \beta, \sin \beta, -\cos \alpha \cdot \cos \beta)$$

α	β	$\mathbf{t}(\alpha, \beta)$	Annex A says
0°	0°	$(0, 0, -1)$ — straight up	neutral position ✓
90°	0°	$(-1, 0, 0)$ — backward	force to the back ✓
0°	-90°	$(0, -1, 0)$ — left	force to the left ✓

Stated assumption: the exterior servo is the outer gimbal (that is what the gyro photo shows). The three single-angle figures cannot distinguish the order; if reversed, only the two per-pod angle formulas swap — nothing else changes.

1.2 · The accounting: five equations, six unknowns

Pod forces $f = T \cdot t(\alpha, \beta)$ at $r_f = (+L_f, 0, 0)$ and $r_r = (-L_r, 0, 0)$; moments at G via $M = r \times f$. The demand gives five equations:

$$\begin{array}{lll} F_x = f_{x,f} + f_{x,r} & \text{(axial)} & \\ F_z = f_{z,f} + f_{z,r} & \text{(vertical pair)} & M_y = -L_f \cdot f_{z,f} + L_r \cdot f_{z,r} \\ 0 = f_{y,f} + f_{y,r} & \text{(no side force)} & M_z = +L_f \cdot f_{y,f} - L_r \cdot f_{y,r} \end{array}$$

Both pods sit on the centerline, so $M_x \equiv 0$ falls out for free — consistent with roll being uncontrolled. Six unknowns minus five equations leaves one free degree of freedom: the axial split, resolved by splitting F_x equally (symmetric; exactly minimum-peak-thrust when the pods carry equal transverse loads). The closed form, with $L = L_f + L_r$:

$$\begin{array}{ll} f_{x,f} = f_{x,r} = F_x' / 2 & \\ f_{z,f} = (L_r \cdot F_z' - M_y') / L & f_{z,r} = (L_f \cdot F_z' + M_y') / L \\ f_{y,f} = +M_z' / L & f_{y,r} = -M_z' / L \quad \text{(pure yaw couple)} \end{array}$$

Physics check: an upward front thrust ($f_{z,f} < 0$, z is down) alone gives $M_y > 0$ — nose up. A rightward front force gives $M_z > 0$ — nose right. Both correct. Per pod, the inversion back to actuator space is exact and iteration-free:

$$T = \|f\| \quad \beta = \arcsin(f_y / T) \quad \alpha = \operatorname{atan2}(-f_x, -f_z)$$

1.3 · Constraint 1 — normalized, unit-free demands

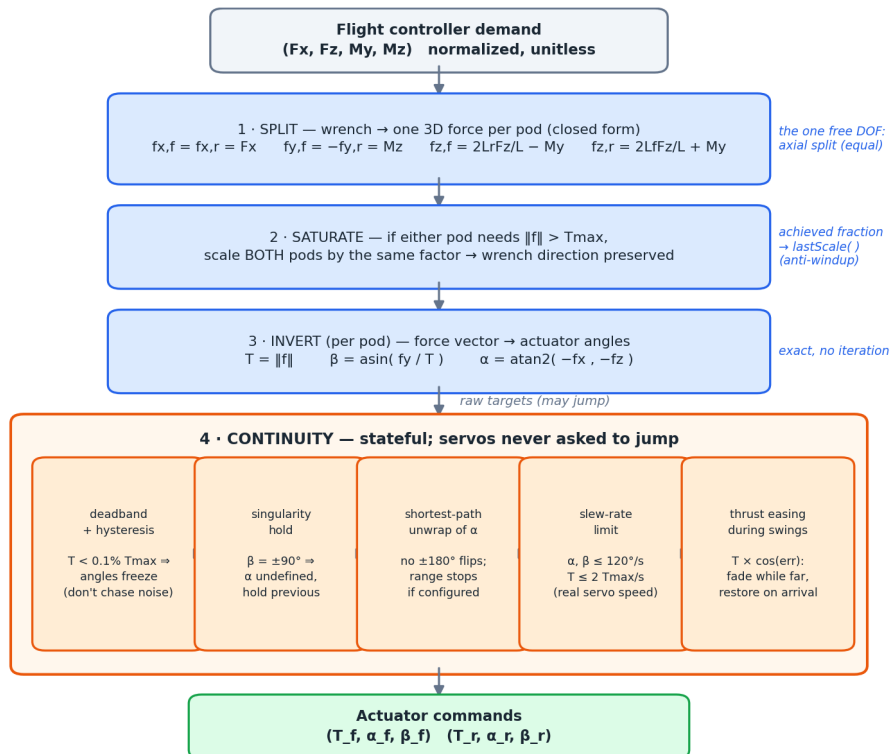
The allocator declares its own scaling, so a unit command on any axis means «the most this airframe can do on that axis»:

- $|F_x| = 1$ or $|F_z| = 1 \Leftrightarrow$ both motors at full thrust along that axis (scale $2 \cdot T_{\max}$); thrust is expressed as a fraction of $T_{\max}$.
- $|M_y| = 1$ or $|M_z| = 1 \Leftrightarrow$ the full antisymmetric pair (scale $(L_f + L_r) \cdot T_{\max}$).
- Combined demands can exceed the per-pod envelope $\|f\| \leq T_{\max}$ (the feasible set is coupled, not a unit box). Then both pods are scaled by one factor: the wrench shrinks but keeps its direction — saturation never invents a moment the controller didn't ask for. The achieved fraction is reported back (lastScale) so upstream integrators can anti-windup against reality.

1.4 · Constraint 2 — no abrupt servo motion

The inversion is memoryless; continuity is a stateful post-processing layer, stage 4 of the pipeline below. The guiding rule, learned the hard way on a simulator: any discontinuity fed to a vectoring actuator under load becomes a transient disturbance — sweep slowly, with thrust faded, and restore on arrival. A full thrust reversal becomes: fade $\rightarrow$ sweep 180° at servo rate $\rightarrow$ restore (≈ 1.5 s).

1.5 · The whole allocator on one page



Stages 1–3: pure stateless algebra. Stage 4 owns all memory. NaN/Inf demands are rejected at the door — the last command holds.

1.6 · Task 2 — the C++ deliverable

file	what it is
src/ControlAllocator.hpp	the class (header-only C++17, no deps): allocate(Wrench) → ActuatorCommand; forward() model; state(), lastScale()
src/demo.cpp	the demonstration script: 8 steady demands, config variants (asymmetric arms, servo hard stops), NaN robustness, reversal & zero-thrust showcases
src/tests.cpp	10 property tests — exact reconstruction (1000 random demands, symmetric + asymmetric geometry), saturation colinearity, slew compliance over 20 000 adversarial ticks, hand-computed inversion check, NaN recovery, noise freeze, hard stops, easing effect

```

g++ -std=c++17 -O2 src/demo.cpp -o demo && ./demo
g++ -std=c++17 -O2 src/tests.cpp -o tests && ./tests # → ALL TESTS PASSED
  
```

Demo output (excerpt) — every case prints the command and the wrench rebuilt by the forward model:

```

climb (Fz=-0.6)      front: T=0.600 alpha= +0.0° beta= +0.0°   rear: T=0.600 alpha= +0.0° beta= +0.0°
cruise+climb+yaw    front: T=0.640 alpha= -45.0° beta=+27.9°   rear: T=0.640 alpha= -45.0° beta=-27.9°
saturating demand   produced F=(+0.53,-0.53) M=(+0.27,+0.27) → 53%, direction kept
reversal Fx +0.8→-0.8: thrust fades to 0, alpha sweeps -90°→+90° at 120°/s, thrust returns
  
```

Design choices an interviewer may probe:

- Closed form, not a QP — for this plant it is the pseudo-inverse: same answer, fixed WCET, saturation handled explicitly instead of emerging from a solver.
- Stateful class — continuity is a property between commands; the allocator remembers the servo pose.
- Real-time honest — no heap, no STL containers, noexcept, O(1); double precision, mechanically portable to float.

Exercise 2 — Mission mode

2.1 · Q1 — why it drifts off the line

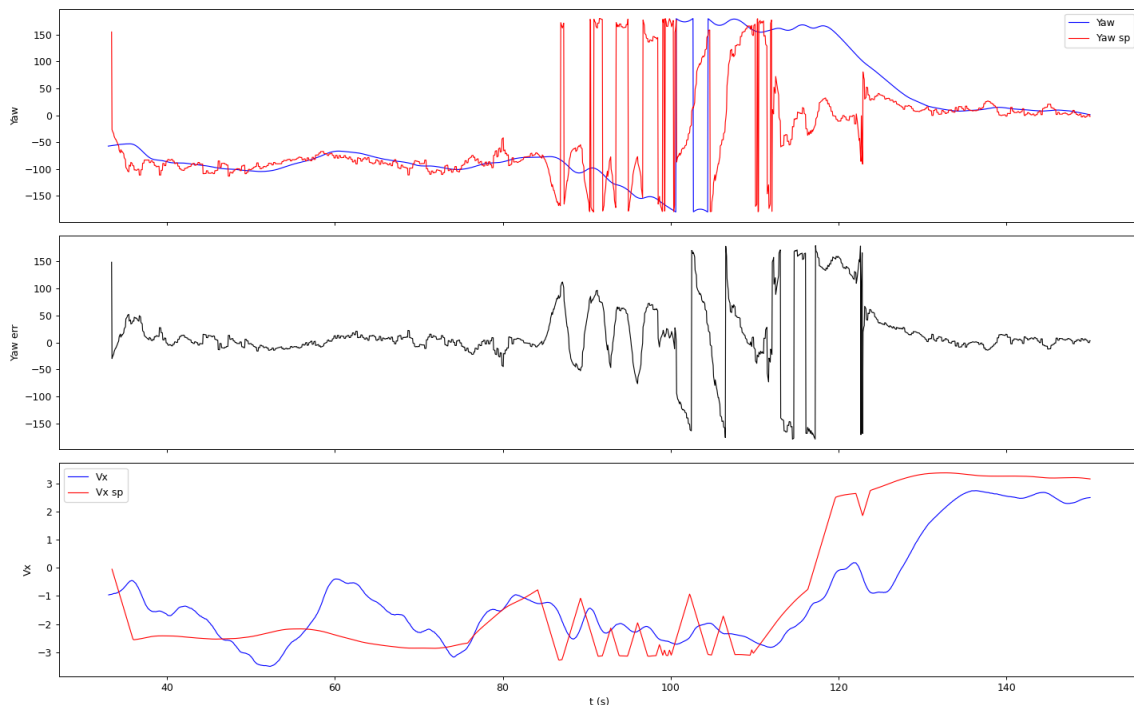
Everything below is measured from the Annex B log (365 s: altitude → mission 33–263 s → position → altitude; every number reproduces via analysis/extract_log.py).

Physical

- No lateral force ($F_y = 0$) — pointing the hull is the only lateral authority the vehicle has.
- Wind $\approx$ half the airspeed. Cruise runs at 2.24 m/s against a 3.0 m/s setpoint; in position mode gusts push the ship backwards to -2.4 m/s.
- Slender-body heading hunt. Yaw error: RMS 10° , peaks 41° , ~ 30 s period. Each degree of nose error at cruise ≈ 4 cm/s of sideways velocity → 0.4–1.4 m/s of drift rate.
- Huge inertia, low yaw authority. The waypoint turnaround takes 45 s for 261° ($\sim 5.8^\circ/\text{s}$) — long stretches flown off-course.

Algorithmic

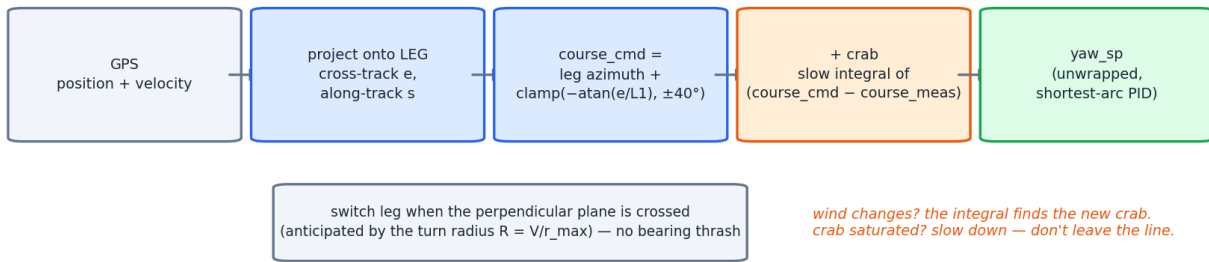
- The PID tracks heading, not course. It tracks its setpoint reasonably well — and the ship still drifts, because in wind the velocity vector is not where the nose points.
- Bearing-chasing guidance converges to the point, not the line — the classic pursuit curve, bowing downwind; cross-track error is never penalized.
- $\pm 180^\circ$ wrap bug: the yaw setpoint flips 22 times in the log when the desired heading sits near south; the vehicle winds through full circles chasing it.
- The loop cannot even see the error. The log has no lateral position, no v_y , no course, no cross-track, no actuator commands — the drift is only inferable. (That is the honest answer to whether Annex B «contains enough information»: not quite — and that gap is part of the problem. Altitude and pitch meanwhile track fine: the problem lives entirely in the horizontal plane.)



Annex B, $t = 33\text{--}150$ s: first leg flown in reverse (v_x setpoint ≈ -2.3 m/s), then the turnaround — the yaw setpoint (red) thrashes across $\pm 180^\circ$ while the vehicle grinds through a 261° rotation in 45 s.

2.2 · Q2 — control the track, not the nose

Realistic with this actuator set: bounded corrections, no wind sensor, no perfect cancellation — the crab angle emerges from integral action.



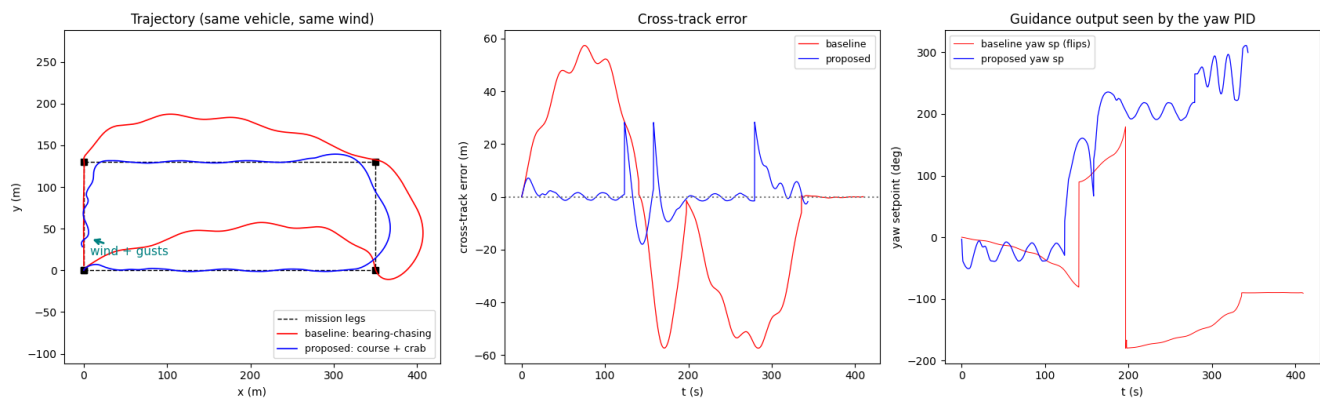
- Measure what matters: project GNSS position onto the current leg → cross-track error e . Pure software; the sensors already exist.
- Command course: $\text{course_cmd} = \text{leg azimuth} + \text{a bounded correction (L1-style)}$. Boundedness is the honesty: in wind beyond the achievable crab the residual drift is bounded and known.
- Let the integral find the crab: $\text{yaw_sp} = \text{course_cmd} + \text{crab}$, $\text{crab} = \text{slow integral of the course error}$. Steady wind ⇒ steady crab, no wind sensor needed.
- Fix the yaw pipeline while at it: shortest-arc errors + an unwrapped setpoint remove the 22 wrap flips for free; switching by perpendicular plane (with turn anticipation $R = V/r_{\max}$) removes the close-range bearing thrash.
- Adapt speed: when the needed crab saturates, slow the along-track setpoint instead of abandoning the line.
- Log e , course and actuator commands — to close the loop and to make the next flight debuggable.

2.3 · The proposal, measured

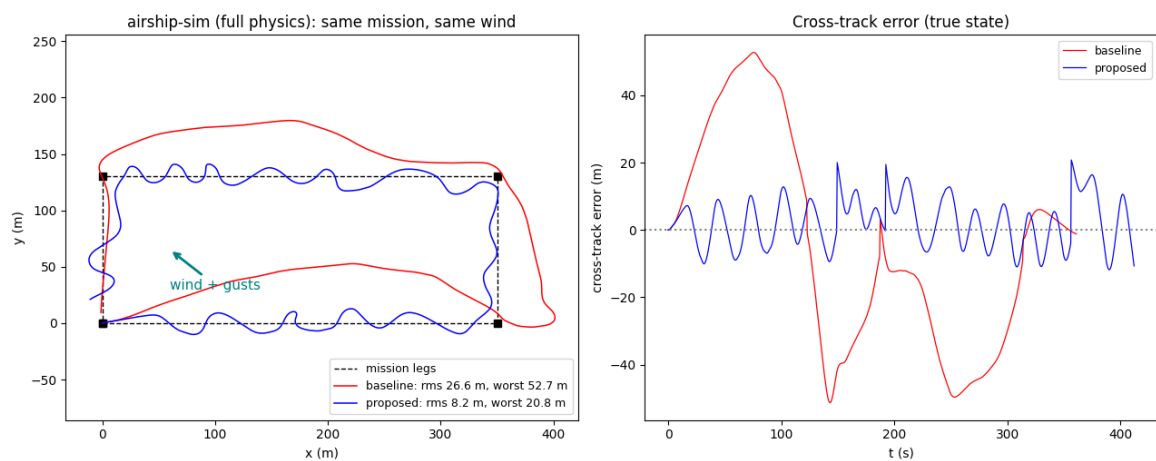
Both guidance laws flew the same missions in the same wind, twice: a minimal planar model, and my own full airship simulator (twin vectoring pods fore/aft, hull weathervane aerodynamics, actuator lags, gusts + turbulence, noisy sensors; the Exercise 1 allocation with $F_y = 0$ respected by construction — yaw is a pure couple). Metrics are whole-leg, no transient exclusions:

	cross-track RMS	worst	mission time
planar — baseline	29.8 m	57.4 m	410 s
planar — proposed	7.5 m	28.4 m	343 s
full sim — baseline (4 wind seeds)	23.4 – 27.0 m	51.6 – 64.3 m	~360 s
full sim — proposed (4 wind seeds)	8.2 – 9.0 m	20.8 – 30.0 m	410 – 550 s

- 2.6–3.2× less RMS cross-track, 1.7–2.7× smaller worst case, mission completed in every run.
- Honest costs: 14–52% more mission time (crabbing against wind at 40–60% of airspeed spends airspeed — the baseline is «faster» because it surrenders to the drift), and a bounded residual: ± 10 –17 m breathing on crosswind legs, 23–30 m at gust peaks on the slow upwind leg, where course observability drops.



Planar model: same vehicle, same wind. The baseline (red) bows downwind on every leg; the proposed guidance (blue) holds the line and hands the yaw PID a continuous setpoint (right panel).



Full simulator, seed 42. The proposed residual is just breathing around the line, not a systematic bow.

2.4 · Using the code (run · read · modify)

Exercise 2 ships as two runnable comparisons plus the log-analysis script. Python pieces need numpy + matplotlib; the simulator piece needs Node ≥ 22 inside the simulator repo (branch ex2-guidance).

```
# 1) planar demo – self-contained, ~5 s
cd analysis && python guidance_demo.py          # prints metrics, writes guidance_demo.png

# 2) full-simulator comparison – inside the simulator repo
node tests/ex2-guidance.mjs                    # default wind seed 42
EX2_SEED=7 node tests/ex2-guidance.mjs        # any other wind realization
python analysis/plot_sim_validation.py analysis/ex2-results.json

# 3) every Annex B number quoted in section 2.1
python analysis/extract_log.py mission_hylight.html
```

What to change, and where:

knob	planar (guidance_demo.py)	simulator (ex2-guidance.mjs)
mission legs	WAYPOINTS, ACCEPT_R	WPTS, ACCEPT_R, Z_HOLD
wind & gusts	WIND_BASE, GUST_AMP, GUST_T	p.windMean, p.windDir, p.gustAmp, p.turb
vehicle agility	R_MAX, R_TAU, KP_YAW, V_CRUISE	R_DES_MAX, K_RATE, V_CRUISE
guidance tuning	L1 = 25 m, crab gain 0.35, clamp $\pm 35^\circ$	L1_DIST, K_CRAB (0.12 — see note), CORR_MAX, CRAB_MAX
which law flies	run('baseline' 'proposed')	same, in the modes loop
wind realization	Plant(seed=...)	EX2_SEED=... env var

Two transferable tuning notes. (1) The crab integrator must be slow relative to the heading loop: in the full simulator $K_{CRAB} = 0.35$ limit-cycled against the heavy, integral-carried yaw loop with a ~ 100 s period; 0.12 is stable (the planar model's faster heading loop tolerates 0.35). (2) Metrics are whole-leg with no «settling» exclusions — an earlier filtered metric turned out to trim transients only from the proposed run, and was removed. The comparison pays for its own corner transients.

Everything in this document regenerates from the solution repository

(github.com/kingsteagall/hylight-gnc-test): src/ (Exercise 1), analysis/ (Exercise 2 + log), deliverable/ (this PDF). Property tests: ./tests → ALL TESTS PASSED.